Типы
Классы типов

data

Алгебраические типы данных. Перечисление

```
data Colour = Red | Green | Blue | White | Black
```

data

Алгебраические типы данных. Перечисление

```haskell
data Colour = Red | Green | Blue | White | Black
```

```haskell
whatIsTheColor :: Colour → String
whatIsTheColor Red = "Color is Red"
whatIsTheColor Green = "Color is Green"
whatIsTheColor Blue = "Color is Blue"
whatIsTheColor White = "Color is White"
whatIsTheColor Black = "Color is Black"
whatIsTheColor _ = "🤨" -- Pattern match is redundant
```

data

Алгебраические типы данных. Перечисление

```haskell
data Colour = Red | Green | Blue | White | Black
```

```haskell
whatIsTheColour :: Colour → String
whatIsTheColour Red = "Colour is Red"
whatIsTheColour Green = "Colour is Green"
whatIsTheColour Blue = "Colour is Blue"
whatIsTheColour White = "Colour is White"
-- Pattern match(es) are non-exhaustive:
-- Patterns of type ‘Colour’ not matched: Black
-- whatIsTheColour Black = "Colour is Black"
```

data

Алгебраические типы данных. Перечисление

```haskell
data Colour = Red | Green | Blue | White | Black
```

```haskell
whatIsTheColour :: Colour → String
whatIsTheColour Red = "Colour is Red"
whatIsTheColour Green = "Colour is Green"
whatIsTheColour Blue = "Colour is Blue"
whatIsTheColour White = "Colour is White"
-- Pattern match(es) are non-exhaustive:
-- Patterns of type ‘Colour’ not matched: Black
-- whatIsTheColour Black = "Colour is Black"
```

- Аналог enum
- Sum type

data

Алгебраические типы данных. Произведение

```
--              имя конструктора
--    имя типа      |
data Point2D = CreatePoint Double Double
--                             |     |
--                            типы полей
```

data

Алгебраические типы данных. Произведение

```haskell
data Point2D = CreatePoint Double Double
```

```haskell
getX :: Point2D → Double
getX (CreatePoint x _) = x

getY :: Point2D → Double
getY (CreatePoint _ y) = y
```

data

Алгебраические типы данных. Произведение

```haskell
data Point2D = CreatePoint Double Double
```

```haskell
getX :: Point2D → Double
getX (CreatePoint x _) = x

getY :: Point2D → Double
getY (CreatePoint _ y) = y
```

```
ghci> :t CreatePoint
CreatePoint :: Double → Double → Point2D

ghci> getX $ CreatePoint 4 7
4.0
```

data

Алгебраические типы данных. Произведение

```
data Point2D a = Point2D a a
```

data

Алгебраические типы данных. Произведение

```haskell
data Point2D a = Point2D a a
```

```haskell
getX :: Point2D a → a
getX (Point2D x _) = x

getY :: Point2D a → a
getY (Point2D _ y) = y
```

data

Алгебраические типы данных. Произведение

```haskell
data Point2D a = Point2D a a
```

```haskell
distance :: Point2D a → Point2D a → a
distance (Point2D x1 y1) (Point2D x2 y2) =
    sqrt $ (x2 - x1) ^ 2 + (y2 - y1) ^ 2
```

data

Алгебраические типы данных. Произведение

```haskell
data Point2D a = Point2D a a
```

```haskell
distance :: Point2D a → Point2D a → a
distance (Point2D x1 y1) (Point2D x2 y2) =
    sqrt $ (x2 - x1) ^ 2 + (y2 - y1) ^ 2
```

Заработает ли?

data

Алгебраические типы данных. Произведение

```haskell
data Point2D a = Point2D a a
```

```haskell
-- distance :: Point2D a → Point2D a → a
distance :: Point2D Double → Point2D Double → Double
distance (Point2D x1 y1) (Point2D x2 y2) =
    sqrt $ (x2 - x1) ^ 2 + (y2 - y1) ^ 2
```

data

Алгебраические типы данных. Сумма произведений

```
data Point a = Point2D a a | Point3D a a a
```

data

Алгебраические типы данных. Сумма произведений

```haskell
data Point a = Point2D a a | Point3D a a a
```

```haskell
toList :: Point a → [a]
toList (Point2D x y) = [x, y]
toList (Point3D x y z) = [x, y, z]
```

data

Алгебраические типы данных. Сумма произведений

```haskell
data Point a = Point2D a a | Point3D a a a
```

```haskell
toList :: Point a → [a]
toList (Point2D x y) = [x, y]
toList (Point3D x y z) = [x, y, z]
```

```haskell
distance :: Point Double → Point Double → Double
distance p1 p2 = sqrt . sum . map (^2) $ zipWith (-) (toList p2) (toList p1)
```

data

Алгебраические типы данных. Сумма произведений

```haskell
data Point a = Point2D a a | Point3D a a a
```

```haskell
toList :: Point a → [a]
toList (Point2D x y) = [x, y]
toList (Point3D x y z) = [x, y, z]
```

```haskell
distance :: Point Double → Point Double → Double
distance p1 p2 = sqrt . sum . map (^2) $ zipWith (-) (toList p2) (toList p1)
```

```
ghci> distance (Point2D 0 0) (Point2D 4 3)
5.0
```

data

Алгебраические типы данных. Сумма произведений

```haskell
data Point a = Point2D a a | Point3D a a a
```

```haskell
toList :: Point a → [a]
toList (Point2D x y) = [x, y]
toList (Point3D x y z) = [x, y, z]
```

```haskell
distance :: Point Double → Point Double → Double
distance p1 p2 = sqrt . sum . map (^2) $ zipWith (-) (toList p2) (toList p1)
```

```
ghci> distance (Point2D 0 0) (Point2D 4 3)
5.0
ghci> distance (Point2D 0 0) (Point3D 4 3 0)
5.0
```

← Это плохо и свидетельствует о неудачном дизайне типа!

data

Алгебраические типы данных. Сумма произведений

```haskell
data Point a = Point2D a a | Point3D a a a
```

```haskell
distance :: Point Double → Point Double → Double
distance p1@(Point2D _ _) p2@(Point2D _ _ ) =
    sqrt . sum . map (^2) $ zipWith (-) (toList p2) (toList p1)
distance p1@(Point3D _ _ _) p2@(Point3D _ _ _) =
    sqrt . sum . map (^2) $ zipWith (-) (toList p2) (toList p1)
distance _ _ = error("Haha, this is partial function now")
```

```
ghci> distance (Point2D 0 0) (Point3D 4 3 0)
*** Exception: Haha, this is partial function now
```

data

Алгебраические типы данных. Сумма произведений

```haskell
data Point a = Point2D a a | Point3D a a a
```

```haskell
distance :: Point Double → Point Double → Double
distance p1@(Point2D _ _) p2@(Point2D _ _ ) =
    sqrt . sum . map (^2) $ zipWith (-) (toList p2) (toList p1)
distance p1@(Point3D _ _ _) p2@(Point3D _ _ _) =
    sqrt . sum . map (^2) $ zipWith (-) (toList p2) (toList p1)
distance _ _ = error("Haha, this is partial function now")
```

```
ghci> distance (Point2D 0 0) (Point3D 4 3 0)
*** Exception: Haha, this is partial function now
```

Избегайте частичных функций!!!!

Конструкторы и записи

```
data Student = Student
  { studentId   :: Int
  , studentName :: String
  , studentYear :: Int
  }
```

```
ghci> studentName (Student 1 "Denis" 4)
"Denis"
```

Конструкторы и записи

```haskell
data Student = Student
  { studentId   :: Int
  , studentName :: String
  , studentYear :: Int
  }
```

= (почти)

```haskell
data Student = Student Int String Int

studentId :: Student → Int
studentId (Student sId _ _) = sId

studentName :: Student → String
studentName (Student _ name _) = name

studentYear :: Student → Int
studentYear (Student _ _ year) = year
```

```
ghci> studentName (Student 1 "Denis" 4)
"Denis"
```

Конструкторы и записи

```haskell
data Student = Student
  { studentId   :: Int
  , studentName :: String
  , studentYear :: Int
  }
```

```haskell
-- синтаксис копирования с частичным обновлением
moveNextYear :: Student → Student
moveNextYear s = s { studentYear = studentYear s + 1 }
```

Конструкторы и записи

```haskell
data Student = Student
  { studentId   :: Int
  , studentName :: String
  , studentYear :: Int
  }
```

```haskell
-- Pattern matching по определенным полям
isFirstYear :: Student → Bool
isFirstYear Student { studentYear = 1 } = True
isFirstYear _                           = False
```

Конструкторы и записи

```
distance :: Point Double → Point Double → Double
distance p1@(Point2D _ _) p2@(Point2D _ _ ) =
    sqrt . sum . map (^2) $ zipWith (-) (toList p2) (toList p1)
distance p1@(Point3D _ _ _) p2@(Point3D _ _ _) =
    sqrt . sum . map (^2) $ zipWith (-) (toList p2) (toList p1)
distance _ _ = error("Haha, this is partial function now")
```

VS

```
distance :: Point Double → Point Double → Double
distance p1@Point2D{} p2@Point2D{} =
    sqrt . sum . map (^2) $ zipWith (-) (toList p2) (toList p1)
distance p1@Point3D{} p2@Point3D{} =
    sqrt . sum . map (^2) $ zipWith (-) (toList p2) (toList p1)
distance _ _ = error("Haha, this is partial function now")
```

Maybe и Either

```haskell
-- либо значения нет, либо есть значение типа a
data Maybe a = Nothing | Just a

-- либо значение типа a, либо значение типа b
data Either a b = Left a | Right b
```

Maybe и Either

```
ghci> lookup 2 [(1, "One"), (2, "Two")]
Just "Two"
ghci> lookup 0 [(1, "One"), (2, "Two")]
Nothing
```

```
studentLabel :: Maybe Student → String
studentLabel Nothing  = "No student no label"
studentLabel (Just s) = studentName s
-- studentLabel (Just (Student _ name _)) = name
```

Maybe и Either

```haskell
data MonthError = MonthTooSmall | MonthTooBig

month :: Int → Either MonthError Int
month n
  | n < 1      = Left MonthTooSmall
  | n > 12     = Left MonthTooBig
  | otherwise = Right n
```

```
ghci> month 0
Left MonthTooSmall

ghci> month 13
Left MonthTooBig

ghci> month 8
Right 8
```

type

```
type Phone = String
type Name = String
type Password = String

register :: Phone → Name → Password → String
register phone name pass = "Registered with phone [" ++ phone ++
"], name [" ++ name ++ "], pass [" ++ pass ++"]"
```

type

```
type Phone = String
type Name = String
type Password = String

register :: Phone -> Name -> Password -> String
register phone name pass = "Registered with phone [" ++ phone ++
"], name [" ++ name ++ "], pass [" ++ pass ++"]"
```

Плюсы:

- Можно использовать для выражения намерения
- Бесплатно (в рантайме)

Минусы:

- Отсутствие типизации = ошибки из-за человека

type

```haskell
type Phone = String
type Name = String
type Password = String

register :: Phone -> Name -> Password -> String
register phone name pass = "Registered with phone [" ++ phone ++
"], name [" ++ name ++ "], pass [" ++ pass ++"]"
```

```
ghci> register "Kre4" "myAwesomePass" "+79880000080"
"Registered with phone [Kre4], name [myAwesomePass], pass [+79880000080]"
```

newtype

```haskell
newtype Phone = Phone String
newtype Name = Name String
newtype Password = Password String

register :: Phone → Name → Password → String
register (Phone ph) (Name n) (Password ps) =
  "Registered with phone [" ++ ph ++ "], name [" ++ n ++ "], pass [" ++ ps ++ "]"
```

```
ghci> let phone = Phone "+79880000080"
ghci> let name = Name "Kre4"
ghci> let pass = Password "myAwesomePass"
ghci> register phone name pass
"Registered with phone [+79880000080], name [Kre4], pass [myAwesomePass]"
ghci> register name pass phone
    * Couldn't match expected type `Phone' with actual type `Name'
    * Couldn't match expected type `Name' with actual type `Password'
    * Couldn't match expected type `Password' with actual type `Phone'
```

`type` vs `newtype` vs `data`

Критерий	`type`	`newtype`	`data`
Суть концепции	Псевдоним для удобства и читаемости кода.	Тонкая защитная обертка над **одним** существующим типом.	Создание абсолютно новой структуры данных с нуля.
Количество конструкторов	**0** (нет своих конструкторов)	**Строго 1**	**Любое**
Количество полей	**0** (нет своих полей)	**Строго 1**	**Любое**
Оверхед в рантайме	**0** - (исчезает при компиляции)	**0%** - (компилятор полностью стирает обертку)	**Есть** (реальный объект в куче)
Проверка компилятором	**Слабая**	**Строгая**	**Строгая**

Классы типов

```
data Colour = Red | Green | Blue | White | Black
```

```
ghci> Red == Red
    * No instance for `Eq Colour' arising
from a use of `=='
```

Классы типов. Eq

```
ghci> :i Eq
class Eq a where
  (==) :: a → a → Bool
  (/=) :: a → a → Bool
  {-# MINIMAL (==) | (/=) #-}
        -- Defined in `GHC.Classes'
instance Eq Integer -- Defined in `GHC.Num.Integer'
instance Eq Bool -- Defined in `GHC.Classes'
instance Eq Char -- Defined in `GHC.Classes'
instance Eq Double -- Defined in `GHC.Classes'
instance Eq Float -- Defined in `GHC.Classes'
instance Eq Int -- Defined in `GHC.Classes'
```

Классы типов. Eq

```
data Colour = Red | Green | Blue | White | Black
```

```
instance Eq Colour where
    Red   == Red   = True
    Green == Green = True
    Blue  == Blue  = True
    White == White = True
    Black == Black = True
    _     == _     = False
```

Классы типов. Eq

```haskell
data Colour = Red | Green | Blue | White | Black
```

```haskell
instance Eq Colour where
    Red   == Red   = True
    Green == Green = True
    Blue  == Blue  = True
    White == White = True
    Black == Black = True
    _     == _     = False
```

```
ghci> Red == Red
True
ghci> Red /= Black
True
```

Классы типов. Eq

```haskell
data Point2D a = Point2D a a
```

```haskell
instance (Eq a) ⇒ Eq (Point2D a) where
    (Point2D x1 y1) == (Point2D x2 y2) = x1 == x2 && y1 == y2
```

```
ghci> Point2D 0 0 == Point2D 0 0
True
ghci> Point2D 0 0 == Point2D 4 2
False
```

Классы типов. Eq

```haskell
data Colour = Red | Green | Blue | White | Black
    deriving (Eq)

data Point2D a = Point2D a a
    deriving (Eq)
```

Классы типов. Show

```
ghci> Red
    * No instance for `Show Colour' arising from a use of
`print'
```

Классы типов. Show

```
ghci> Red
    * No instance for `Show Colour' arising from a use of
`print'
```

```
instance Show Colour where
    show Red   = "Red"
    show Green = "Green"
    show Blue  = "Blue"
    show White = "White"
    show Black = "Black"
```

```
data Colour = …
    deriving (Eq, Show)
```

```
ghci> Red
Red
```

Классы типов. Ord

```
type Year = Int
type Month = Int
type Day = Int
data Date = Date Year Month Day
    deriving Eq
```

```
ghci> :i Ord
class Eq a ⇒ Ord a where
  compare :: a → a → Ordering
  (<) :: a → a → Bool
  (⩽) :: a → a → Bool
  (>) :: a → a → Bool
  (⩾) :: a → a → Bool
  max :: a → a → a
  min :: a → a → a
  {-# MINIMAL compare | (⩽) #-}
```

Классы типов. Ord

```haskell
type Year = Int
type Month = Int
type Day = Int
data Date = Date Year Month Day
    deriving Eq
```

```haskell
instance Ord Date where
    compare (Date y1 m1 d1) (Date y2 m2 d2) =
        compare (y1, m1, d1) (y2, m2, d2)
```

```
ghci> Date 2026 10 10 < Date 2026 10 15
True
```

Классы типов. class

```
class HasArea a where
  area :: a → Double
```

Классы типов. class

```haskell
class HasArea a where
  area :: a → Double
```

```haskell
data Shape = Circle Double | Rectangle Double Double
  deriving (Eq, Show)
```

```haskell
instance HasArea Shape where
  area (Circle r) = pi * r ^ 2
  area (Rectangle w h) = w * h
```

Классы типов. class

```haskell
class HasArea a where
  area :: a → Double
```

```haskell
data Shape = Circle Double | Rectangle Double Double
  deriving (Eq, Show)
```

```haskell
instance HasArea Shape where
  area (Circle r) = pi * r ^ 2
  area (Rectangle w h) = w * h
```

```haskell
printArea :: HasArea a ⇒ a → String
printArea x = "Calculated area = " ++ (show $ area x)
```

```
ghci> printArea $ Rectangle 5 4
"Calculated area = 20.0"
```

Классы типов. class

```haskell
data Shape3D = Sphere Double | Cube Double
  deriving (Eq, Show)

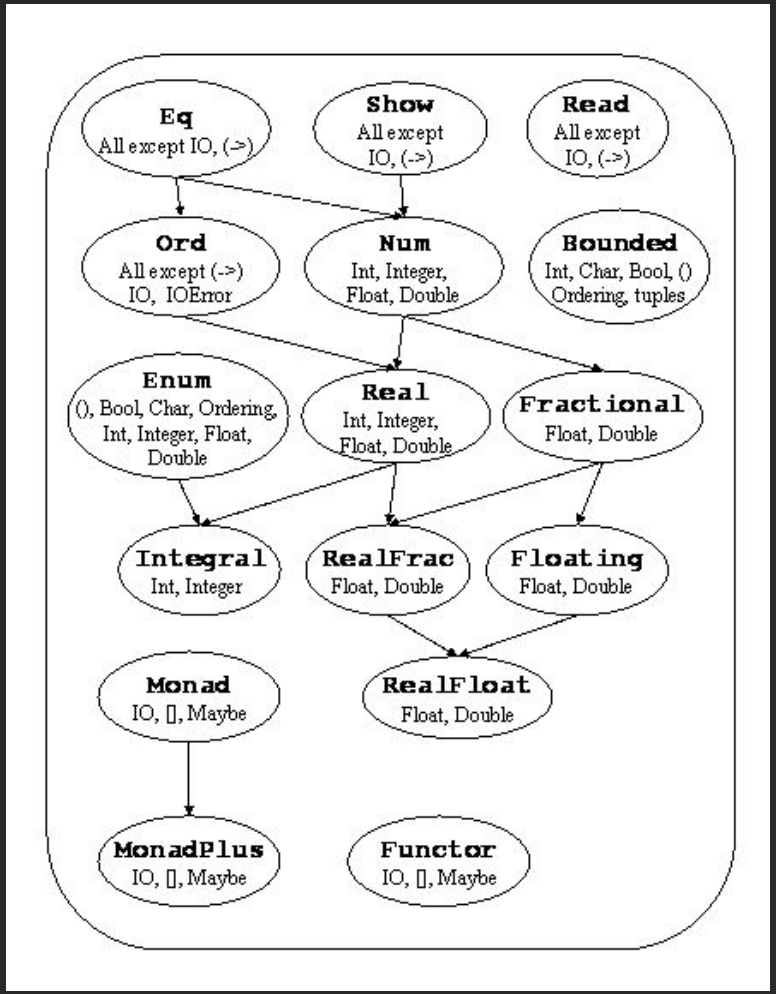
instance HasArea Shape3D where
  area (Cube s) = 6 * s ^ 2
  area (Sphere r) = 4 * pi * r ^ 2
```

Классы типов. class

```haskell
data Shape3D = Sphere Double | Cube Double
  deriving (Eq, Show)

instance HasArea Shape3D where
  area (Cube s) = 6 * s ^ 2
  area (Sphere r) = 4 * pi * r ^ 2
```

```
ghci> printArea $ Cube 5
"Calculated area = 150.0"
```



https://www.haskell.org/onlinereport/basic.html



ВСЁ!